Comparative Analysis of IaaS, PaaS, and SaaS of GCP, AWS and IBM Cloud

Himanshu Jhawar (hj2713)

Abstract

This report presents a hands on evaluation of cloud computing service models across three major vendors: IBM Cloud, Google Cloud Platform (GCP), and Amazon Web Services (AWS). As a first-time explorer of comprehensive cloud services beyond basic virtual machine creation, I examined one Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS) offering from each provider. The study focuses on usability from a Mac user's perspective, service creation performance, and the learning curve associated with each platform. Key observations include significant variation in onboarding complexity, with IBM Cloud requiring additional verification steps for student accounts, while GCP and AWS offered more streamlined registration processes. Performance metrics reveal distinct provisioning characteristics across service models, with IaaS generally requiring more configuration time than PaaS or SaaS alternatives.

1 Introduction

1.1 Background and Purpose

Cloud computing has evolved into three primary service models: Infrastructure as a Service (IaaS) providing virtualized computing resources, Platform as a Service (PaaS) offering managed application deployment environments, and Software as a Service (SaaS) delivering ready-to-use applications. This report evaluates comparable services across IBM Cloud, Google Cloud Platform, and Amazon Web Services to understand practical differences in usability, capability, and performance.

1.2 Approach

As an MS Computer Science student with minimal prior cloud experience (basic GCP VM creation in a previous course), I approached this study as a novice user. All interactions were conducted on macOS. I created accounts on all three platforms, noting onboarding differences, then provisioned and tested nine services (three per vendor). Performance was measured through informal timing of service creation and initial configuration.

2 Google Cloud Platform

2.1 Compute Engine (IaaS)

Having previously created GPU-enabled VMs in another course, I was somewhat familiar with the GCP Console. For this exercise, I provisioned a new `e2 medium` instance in the `us central1` region. The instance creation process took approximately 90-100 seconds from clicking "Create" to SSH availability. I utilized the browser based Cloud Shell for terminal access, which eliminated local configuration needs on my Mac.

A notable aspect of the experience was the GPU availability checking mechanism. When attempting to add an NVIDIA T4 GPU to the instance, the system required approximately 30-40 seconds to validate regional quota availability before displaying configuration options. While this delay was understandable for resource verification, it added uncertainty to the workflow. The interface clearly communicated quota limitations, and the eventual provisioning succeeded without error.

Compute Engine provided extensive customization options, including machine type selection, boot disk configuration, and firewall rule management. The browser-based SSH client functioned smoothly, requiring no additional software installation.

2.2 App Engine (PaaS)

For the PaaS evaluation, I deployed a Python Flask application using App Engine Standard environment. While I initially anticipated command-line deployment, I discovered that GCP Console provides substantial GUI support for this process. I created the application through the Console's App Engine section, then used the integrated Cloud Shell (accessible via browser button) to execute deployment commands.

The deployment process required approximately 3 minutes from initialization to live URL availability. The Console displayed real-time build logs, which helped understand the underlying process. App Engine automatically handled SSL certificate provisioning and load balancing configuration, aspects that would require manual setup in an IaaS environment.

The service abstracted infrastructure management effectively. I observed automatic scaling configuration in the Console's "Versions" section, where instance allocation settings were visible but pre-configured with reasonable defaults. This reduced operational decision burden compared to Compute Engine.

2.3 Google Workspace (SaaS)

Evaluating SaaS through Google Workspace provided an interesting contrast to infrastructure services. I created a shared Google Doc and configured sharing permissions, representing typical SaaS consumption patterns. The service was immediately available upon login; no provisioning time was measurable.

The experience highlighted complete abstraction of underlying infrastructure. As a user, I interacted solely with document editing features, with no visibility into servers, scaling, or maintenance windows. Collaboration features functioned as expected, with real-time co-editing and version history accessible through the interface. This represented the highest level of managed service among the nine evaluated, requiring zero configuration for basic functionality.

3 Amazon Web Services

3.1 EC2 (IaaS)

Creating an AWS account involved more extensive form completion than GCP, including detailed contact verification and payment method confirmation. Once registered, I navigated to the EC2 Console to launch a `t2.micro` instance. The launch wizard presented a single configuration screen with multiple collapsible sections: AMI selection, instance type configuration, security group setup, storage configuration, and key pair management.

The instance reached "running" state in approximately 2 minutes, though SSH accessibility required an additional 30-60 seconds for system initialization. Security group configuration presented the steepest learning curve; I needed to explicitly add an inbound rule for SSH (port 22) from my IP address to establish connection. This step, while adding security, required understanding of network concepts that GCP had abstracted behind simpler firewall checkboxes.

The EC2 Console provided dense information presentation, with numerous instance metrics and monitoring options immediately visible. This information richness supported detailed resource management but required more navigation to locate specific configuration options compared to GCP's Compute Engine interface.

3.2 Lambda (PaaS)

AWS Lambda offered a primarily Console-based creation workflow that aligned with my GUI preference. I created a Python 3.11 function using the "Author from scratch" option in the Lambda Console. The process required specifying function name, runtime selection, and architecture choice (x86_64 vs arm64), followed by permission configuration through AWS's IAM service.

Function deployment from the Console took approximately 1 minute. The inline code editor allowed immediate testing through the "Test" tab, which created a configurable test event. I observed that Lambda's event-driven model differed significantly from App Engine's request-based approach; while both abstracted servers, Lambda's function-as-a-service paradigm required understanding of trigger configurations (API Gateway, S3 events, etc.) for practical application deployment.

The monitoring section provided immediate access to CloudWatch metrics, showing invocation counts and duration statistics. This integrated observability represented a strength of the AWS platform, though the breadth of available metrics initially required exploration to interpret meaningfully.

3.3 QuickSight (SaaS)

Amazon QuickSight required separate subscription activation beyond standard AWS account setup. I initiated the Standard edition trial, which provisioned within approximately 2 minutes. The interface presented business intelligence workflows dataset creation, analysis building, and dashboard publishing without infrastructure configuration requirements.

Creating my first analysis from a sample dataset required navigating through data source connection, field selection, and visualization type choices. The learning curve stemmed from BI-specific concepts rather than cloud infrastructure, distinguishing this SaaS experience from infrastructure-oriented services. Response times for query execution and visualization rendering were sub-second for the sample data, suggesting efficient backend resource management.

4 IBM Cloud

4.1 Virtual Servers (IaaS)

IBM Cloud account creation required the most extensive verification among the three providers, particularly for the student account type I selected. The process involved academic email verification and additional documentation review, extending onboarding time compared to GCP and AWS. Once registered, I provisioned a virtual server instance in the Dallas region.

The provisioning process required approximately 3 minutes from configuration submission to active status. The IBM Cloud Console presented instance options through a profile-based selection system, where "balanced," "compute," and "memory" profiles simplified machine type selection compared to AWS's granular instance families. Network configuration was pre-populated with default values, reducing initial decision complexity.

SSH access required downloading an SSH key through the Console, as password authentication was not offered for the selected Ubuntu image. The connection process functioned identically to GCP and AWS once established, with standard OpenSSH compatibility on macOS.

4.2 Cloud Foundry (PaaS)

IBM Cloud Foundry deployment was accessible through both Console and CLI pathways. I utilized the Console's "Create application" flow, selecting a Python buildpack for a sample Flask application. The platform handled dependency installation through detected `requirements.txt` contents, with build process logs visible in real time.

Application deployment required approximately 5 minutes, including buildpack staging and container initialization. The Console provided application status indicators and direct URL access upon completion. I noted that Cloud Foundry's approach emphasized application portability through buildpacks, with the same application structure potentially deployable across different Cloud Foundry instances.

Resource allocation was visible through the "Runtime" section, showing memory and disk quotas. Scaling configuration was available but required explicit adjustment from default single instance deployment, differing from App Engine's automatic scaling defaults.

4.3 Watson Studio (SaaS)

Watson Studio provisioned as a managed service within the IBM Cloud Console's "Resource list." The service instance creation required approximately 2 minutes, after which the Watson Studio interface opened as a separate web application. This two layer navigation (IBM Cloud Console to Watson Studio) differed from fully integrated SaaS experiences like Google Workspace.

Within Watson Studio, I created a new project and explored the Jupyter notebook environment. The interface presented data science workflows data asset management, notebook creation, and model deployment without infrastructure visibility. Notebook initialization required approximately 30 seconds, with compute resources apparently provisioned on demand. The experience emphasized machine learning and data science capabilities, distinguishing it from general productivity SaaS offerings.

5 Comparative Analysis

5.1 Service Model Characteristics

Table 1: Cross-Vendor Comparison of Cloud Service Characteristics

Comparison Point	**Google Cloud**	**AWS**	**IBM Cloud**	**Observation**
IaaS Provision Time	90-100 seconds	∼2.5 minutes	∼3 minutes	Measured variation across platforms
IaaS Security Configuration	Firewall checkboxes	Security group rules	Pre-populated defaults	Different abstraction approaches
Account Verification	Email confirmation	Detailed identity forms	Billing System error but got Student account	Varying verification thoroughness
Console Information Density	Spaced layout	Dense dashboards	Category-organized	Distinct interface philosophies
PaaS GUI Support	Full Console deployment	Full Console deployment	Console and CLI options	All supported GUI workflows
SaaS Activation	Immediate	∼2 minutes	∼2 minutes	Near-instant availability

IaaS services consistently required the most configuration time and technical decision making. EC2's security group setup and Compute Engine's GPU quota verification represented platform-specific complexity additions. PaaS services reduced infrastructure decisions but introduced platform-specific deployment models; App Engine's application structure requirements, Lambda's event-driven paradigm, and Cloud Foundry's buildpack system each required distinct learning. SaaS offerings provided immediate productivity, with Watson Studio and QuickSight requiring domain knowledge (data science, BI) rather than cloud infrastructure expertise.

5.2 Vendor Specific Observations

Account creation experiences varied meaningfully. I utilized an existing GCP account (created during a previous semester for another subject), whereas AWS required more extensive identity verification. IBM Cloud's academic account process involved additional student email validation steps. This process was complicated by initial difficulty locating the student account option. Console interface designs reflected different philosophies: GCP emphasized clean, spaced layouts; AWS presented information-dense dashboards; IBM Cloud organized services through category-based navigation.

Provisioning performance differed across vendors for comparable services. Compute Engine demonstrated fastest IaaS availability (90-100 seconds), followed by EC2 (approximately 2.5 minutes including accessibility delay), then IBM Virtual Servers (3 minutes). These variations likely reflect different initialization processes and resource allocation strategies rather than infrastructure quality differences.

5.3 Usability for GUI Preferred Users

All three vendors supported GUI-based workflows for the evaluated services, contrary to my initial assumption that PaaS would require command-line interaction. GCP's Cloud Shell integration provided seamless browser-based terminal access when needed, preserving GUI workflow continuity. AWS Lambda's Console supported complete function management without CLI necessity. IBM Cloud Foundry offered equivalent Console-based deployment options.

Documentation accessibility varied. GCP's documentation frequently included Console-specific instructions with screenshot references. AWS documentation, while comprehensive, often prioritized CLI examples requiring additional navigation to locate Console procedures. IBM documentation provided adequate guidance but required more cross-referencing between Cloud Foundry and IBM Cloud-specific materials.

6 Conclusion

This evaluation revealed that cloud service selection involves trade-offs between control and convenience that align with specific use cases. IaaS services (Compute Engine, EC2, Virtual Servers) provided maximum flexibility at the cost of operational responsibility, suitable for applications requiring custom environments. PaaS offerings (App Engine, Lambda, Cloud Foundry) balanced abstraction with configurability, appropriate for application-focused development without infrastructure management. SaaS products (Google Workspace, QuickSight, Watson Studio) delivered immediate functionality for specific domains, ideal for end-user productivity and specialized workflows.

For students and new cloud practitioners, GCP demonstrated the most approachable onboarding experience, while AWS provided the broadest service ecosystem. IBM Cloud's strength lay in enterprise-oriented features and academic program integration, albeit with additional initial setup requirements. The progression from IaaS to SaaS across all vendors consistently reduced provisioning complexity and time to functionality, validating the service model taxonomy's practical relevance.

References

[1] Google Cloud. (2024). *Compute Engine Documentation*. https://cloud.google.com/compute/docs

[2] Google Cloud. (2024). *App Engine Documentation*. https://cloud.google.com/appengine/docs

[3] Google. (2024). *Google Workspace Admin Help*. https://support.google.com/a

[4] Amazon Web Services. (2024). *Amazon EC2 User Guide*. https://docs.aws.amazon.com/ec2/

[5] Amazon Web Services. (2024). *AWS Lambda Developer Guide*. https://docs.aws.amazon.com/lambda/

[6] Amazon Web Services. (2024). *Amazon QuickSight User Guide*. https://docs.aws.amazon.com/quicksight/

[7] IBM Cloud. (2024). *Virtual Server Instances Documentation*. https://cloud.ibm.com/docs/vpc

[8] IBM Cloud. (2024). *Cloud Foundry Public Documentation*. https://docs.cloudfoundry.org/devguide/services/application-binding.html

[9] IBM Cloud. (2024). *Watson Studio Documentation*. https://dataplatform.cloud.ibm.com/docs/content/wsj/getting-started/welcome-main.html